Informe: aplicación del token JWT en el sistema de Órdenes de Compra

Mini Proyecto 1 · Día 2 · Investigación sobre cómo autenticar la API con un token y cómo lo aplicamos en el proyecto.

---

1. ¿Qué es un JWT?

Un **JSON Web Token** (JWT, RFC 7519) es una cadena compacta que transporta datos (llamados *claims*) entre dos partes, firmada para que nadie pueda modificarla sin que se note. Tiene tres partes separadas por puntos, cada una codificada en **Base64URL**:

```
encabezado.payload.firma
```

Parte	Contenido	Ejemplo en nuestro proyecto
Header	Tipo de token y algoritmo de firma	`{ "alg": "HS256", "typ": "JWT" }`
Payload	Los *claims*: quién es el usuario y cuándo vence	`{ "sub": "1", "iat": 1793810004, "exp": 1793838804 }`
Firma	`HMAC-SHA256(header + "." + payload, JWT_SECRET)`	43 caracteres Base64URL

- `sub` (*subject*): id del usuario dueño del token.
- `iat` (*issued at*): cuándo se emitió.
- `exp` (*expiration*): cuándo deja de ser válido.

Importante: un JWT firmado no está cifrado. Cualquiera puede decodificar el payload (basta con Base64URL), así que nunca debe llevar datos sensibles. La firma solo garantiza **integridad** y **autenticidad**: que el token lo emitió nuestro servidor y que nadie lo alteró.

Cómo se firma y se verifica

Usamos **HS256** (HMAC con SHA-256), un algoritmo simétrico: la misma clave secreta firma y verifica.

1. **Firmar (login):** el servidor arma header y payload, los codifica y calcula la firma con `JWT_SECRET`.
2. **Verificar (cada petición):** el servidor vuelve a calcular la firma con la misma clave y la compara con la que trae el token. Si alguien cambió un solo carácter del payload (por ejemplo, `sub: "1"` por `sub: "2"`), la firma ya no coincide y el token se rechaza. Después revisa que `exp` no haya pasado.

La alternativa **RS256** usa un par de claves (privada para firmar, pública para verificar). Conviene cuando varios servicios distintos deben verificar tokens sin poder emitirlos. En nuestro caso un solo back end emite y verifica, así que HS256 es suficiente y más simple.

2. Generar y validar el token en Node.js con Express

Usamos la librería **jsonwebtoken** (la más usada en Node.js, mantenida por Auth0).

Generar (`backend/src/controllers/authController.js`, al iniciar sesión):

```
const token = jwt.sign({ sub: String(usuario.id) }, JWT_SECRET, {
  algorithm: 'HS256',
  expiresIn: JWT_EXPIRES_IN, // '8h'
});
```

Validar (`backend/src/middlewares/authMiddleware.js`):

```
datos = jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] });
```

`jwt.verify` comprueba la firma y la expiración, y lanza un error si algo falla. Pasar `algorithms: ['HS256']` es importante: obliga a usar solo ese algoritmo y evita ataques donde un token falso declara `"alg": "none"` o cambia de algoritmo para saltarse la verificación (RFC 8725).

Las contraseñas se guardan con **bcrypt** (`bcryptjs` , costo 10): un hash lento y con *salt* aleatorio, de modo que dos contraseñas iguales producen hashes distintos y probar contraseñas por fuerza bruta resulta muy costoso.

3. Expiración, renovación y qué hacer cuando vence

- **Expiración (`exp`):** un token robado sirve solo hasta que vence. Mientras más corto, menor el daño, pero el usuario debe iniciar sesión más seguido.
- **Refresh token:** el patrón habitual en sistemas grandes combina un *access token* corto (5 a 15 min) con un *refresh token* largo, guardado en una cookie `httpOnly` y registrado en la base de datos. Cuando el access token vence, el front pide uno nuevo con el refresh token sin molestar al usuario. El refresh token se **rota** en cada uso y se puede **revocar** (por ejemplo, al cerrar sesión o si se detecta un robo).
- **Cuando vence:** la API responde **401**. El front borra el token guardado y manda al usuario al login con un aviso.

Decisión del proyecto: token de acceso de **8 horas** (una jornada laboral) y **sin refresh token**. El PDF del día 2 deja fuera de alcance los roles, y el refresh token agrega una tabla, rotación y revocación que no aportan a los CRUDs, que son el foco. La duración se configura con `JWT_EXPIRES_IN` en el `.env` . El front además lee `exp` y cierra la sesión justo cuando vence, sin esperar a que falle una petición.

4. ¿Dónde guardar el token en React?

Opción	Ventajas	Riesgos
localStorage	Persiste al recargar o cerrar la pestaña; fácil de usar	Cualquier script que corra en la página puede leerlo: vulnerable a **XSS**
sessionStorage	Se borra al cerrar la pestaña	El mismo riesgo de XSS; la sesión no se comparte entre pestañas
Memoria (estado de React)	Ningún script externo lo encuentra guardado	Se pierde al recargar la página; obliga a usar refresh token
Cookie `httpOnly`	JavaScript no puede leerla: protegida contra XSS	El navegador la envía sola en cada petición: hay que protegerse de **CSRF** (`SameSite` , token anti-CSRF); requiere configurar CORS con credenciales

- **XSS** (*Cross-Site Scripting*): un atacante logra ejecutar JavaScript en nuestra página y lee lo que haya en `localStorage` .
- **CSRF** (*Cross-Site Request Forgery*): otro sitio hace que el navegador del usuario envíe una petición a nuestra API, y la cookie viaja sola.

Decisión del proyecto: `localStorage` (opción que el PDF menciona explícitamente), con estas medidas para reducir el riesgo de XSS:

- React escapa todo el texto que muestra, y el proyecto no usa `dangerouslySetInnerHTML` ni inserta HTML de los datos.
- El payload solo lleva el id del usuario; no hay correo, nombre ni contraseña.
- El token vence en 8 horas y se borra al cerrar sesión o al recibir un 401.
- Para producción se recomienda además una política de seguridad de contenido (CSP).

Si el sistema pasara a producción con datos reales, la mejora sería mover el token a una cookie `httpOnly` + `Secure` + `SameSite=Strict` y agregar refresh token.

5. El middleware que protege las rutas y el envío desde el front end

El token viaja en el encabezado estándar `Authorization` con el esquema **Bearer** (RFC 6750):

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

Back end (`authMiddleware.js`): se monta antes de todas las rutas de los CRUDs en `routes/index.js`. En cada petición:

1. Lee el encabezado y exige el formato `Bearer <token>`.
2. Verifica la firma y la expiración con `jwt.verify`.
3. Busca al usuario del `sub` en la base de datos (por si la cuenta fue eliminada).
4. Si todo está bien, deja el usuario en `req.usuario` y continúa. Si algo falla, responde **401** `{ "error": "No autorizado" }`.

Quedan públicas solo `GET /api` (estado del servicio), `POST /api/auth/register` y `POST /api/auth/login`.

Front end:

- `services/api.js` tiene una sola función `apiFetch` que usan todos los servicios: agrega el encabezado `Authorization` a cada petición y, si la API responde 401, avisa al `AuthContext`.
- `context/AuthContext.jsx` guarda el token y el usuario actual, y expone `iniciarSesion`, `registrar` y `cerrarSesion`.
- `components/RutaProtegida.jsx` envuelve todo el panel: sin token, redirige a `#/login` y recuerda la pantalla que se pidió, para volver a ella después del login.

6. Buenas prácticas de seguridad aplicadas

Práctica	Cómo se aplica
Clave secreta fuera del código	`JWT_SECRET` vive en `backend/.env`, que está en `.gitignore`; el repositorio solo tiene `.env.example` sin valores. El servidor no arranca si falta o tiene menos de 32 caracteres
Clave fuerte	48 bytes aleatorios generados con `crypto.randomBytes`
Payload mínimo	Solo `sub`, `iat` y `exp`; ningún dato personal
Algoritmo fijo al verificar	`algorithms: ['HS256']`
Expiración	8 horas, configurable

Práctica	Cómo se aplica
Contraseñas	Hash bcrypt con costo 10; mínimo 8 caracteres; nunca se devuelven (scope por defecto que excluye la columna y `toJSON` que la elimina)
Mensaje de login genérico	"Correo o contraseña incorrectos" en ambos casos, para no revelar qué correos tienen cuenta
CORS limitado	Solo se acepta el origen del front (`CORS_ORIGIN`)
HTTPS	En desarrollo se usa `localhost`; en producción la API y el front deben servirse por HTTPS para que el token no viaje en texto plano

7. Cómo se comprobó

La colección de Postman (`postman/ordenes-compra.postman_collection.json`) incluye la carpeta **Autenticación**, que se ejecuta primero:

- Registro correcto (201, sin contraseña en la respuesta), correo repetido (400) y datos inválidos (400).
- Login con contraseña incorrecta y con correo inexistente (401 con el mismo mensaje), y login correcto: guarda el token y comprueba que el payload tenga `exp` y no tenga datos sensibles.
- Perfil con token (200), sin token, con token inválido, con token alterado sin volver a firmarlo y sin el prefijo `Bearer` (todos 401).
- Rutas de los CRUDs sin token (401).

El resto de la colección usa el token del login a nivel de colección.

Fuentes consultadas

1. Jones, M., Bradley, J. y Sakimura, N. (2015). *RFC 7519: JSON Web Token (JWT)*. IETF. https://datatracker.ietf.org/doc/html/rfc7519
2. Jones, M., Bradley, J. y Sakimura, N. (2015). *RFC 7515: JSON Web Signature (JWS)*. IETF. https://datatracker.ietf.org/doc/html/rfc7515
3. Sheffer, Y., Hardt, D. y Jones, M. (2020). *RFC 8725: JSON Web Token Best Current Practices*. IETF. https://datatracker.ietf.org/doc/html/rfc8725
4. Jones, M. y Hardt, D. (2012). *RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage*. IETF. https://datatracker.ietf.org/doc/html/rfc6750
5. Auth0. *Introduction to JSON Web Tokens*. jwt.io. https://jwt.io/introduction
6. Auth0. *jsonwebtoken* (documentación de la librería). https://github.com/auth0/node-jsonwebtoken
7. OWASP. *JSON Web Token Cheat Sheet* (algoritmo, almacenamiento del token y revocación). https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_Cheat_Sheet.html
8. OWASP. *HTML5 Security Cheat Sheet* (sección Local Storage). https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
9. OWASP. *Cross-Site Request Forgery Prevention Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html
10. OWASP. *Password Storage Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
11. MDN Web Docs. *Set-Cookie* (atributos `HttpOnly`, `Secure` y `SameSite`). https://developer.mozilla.org/es/docs/Web/HTTP/Headers/Set-Cookie
12. Express. *Production Best Practices: Security*. https://expressjs.com/en/advanced/best-practice-security.html

13. Wirtz, D. *bcrypt.js* (documentación del paquete `bcryptjs`). https://github.com/dcodeIO/bcrypt.js